

```

"""
HoloKai FastAPI - African First Civilization Platform API
16-Volume Master Library - 185 chunks - Thunder Code Edition

For FEEXARA ⚡ The Thunder Code

Endpoints:
- GET /health
- GET /volumes
- POST /query - semantic search
- POST /chat - RAG with LLM
- POST /thunder - FEEXARA signature query
- GET /random - discovery
"""

from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List, Optional, Dict, Any
import os
import random
import json

# ===== CONFIG =====
CHROMA_PATH = "./holokai_chroma_db"
COLLECTION_NAME = "holokai_african_civilization"
EMBEDDING_PROVIDER = "local" # "local" or "openai"
LOCAL_MODEL = "BAAI/bge-large-en-v1.5"
OPENAI_MODEL = "text-embedding-3-large"
LLM_MODEL = "gpt-4o-mini" # for RAG chat

# ===== APP INIT =====
app = FastAPI(
    title="HoloKai API - African First Civilization Memory Core",
    description="16-Volume Master Library | 185 chunks | 2500+ facts | 10k years | Thunder Code Edition",
    version="16.0.0",
    contact={"name": "FEEXARA - The Thunder Code", "url": "https://holokai.africa"}
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ===== MODELS =====
class QueryRequest(BaseModel):
    query: str
    n_results: int = 5
    region: Optional[str] = None # e.g., "West Africa", "Nile Valley"
    period: Optional[str] = None # e.g., "Ancient", "Classical/Medieval"
    volume: Optional[str] = None # e.g., "V14"

class QueryResponse(BaseModel):
    query: str
    results: List[Dict[str, Any]]
    count: int

class ChatRequest(BaseModel):
    message: str
    n_context: int = 5
    history: Optional[List[Dict[str, str]]] = None

class ThunderRequest(BaseModel):
    intent: str # e.g., "energy", "transformation", "iron", "thunder"
    query: Optional[str] = None

# ===== LOAD CHROMA =====
chroma_client = None
collection = None
embed_model = None

@app.on_event("startup")
async def startup():
    global chroma_client, collection, embed_model
    print("⚡ HoloKai Booting - Thunder Code Loading...")
    try:
        import chromadb
        chroma_client = chromadb.PersistentClient(path=CHROMA_PATH)
        collection = chroma_client.get_collection(COLLECTION_NAME)
        print(f"✓ Chroma loaded: {collection.count()} docs")

        if EMBEDDING_PROVIDER == "local":
            from sentence_transformers import SentenceTransformer
            embed_model = SentenceTransformer(LOCAL_MODEL)
            print(f"✓ Embedding model: {LOCAL_MODEL}")
        except Exception as e:
            print(f"⚠ Chroma not found at {CHROMA_PATH}: {e}")
            print("Run holokai_chroma_pinecone_upload.py first")
            # Create dummy for docs
            collection = None

# ===== ENDPOINTS =====

@app.get("/")
async def root():
    return {
        "name": "HoloKai - African First Civilization Memory Core",
        "version": "16.0.0",
    }

```

```

        "volumes": 16,
        "chunks": collection.count() if collection else 0,
        "thunder_code": "FEEXARA ⚡ Active",
        "endpoints": ["/health", "/volumes", "/query", "/chat", "/thunder", "/random"],
        "docs": "/docs"
    }

@app.get("/health")
async def health():
    return {
        "status": "ok" if collection else "no_db",
        "chroma_path": CHROMA_PATH,
        "count": collection.count() if collection else 0,
        "embedding": EMBEDDING_PROVIDER,
        "thunder": "⚡"
    }

@app.get("/volumes")
async def list_volumes():
    """List all 16 volumes in library"""
    return {
        "volumes": [
            {"id": "V1", "title": "Ancient Foundations", "chunks": 15, "focus": "Egypt, Nubia, Ishango, Maat"},
            {"id": "V2", "title": "Classical Empires", "chunks": 12, "focus": "Ghana, Mali, Songhai, Benin, Kongo"},
            {"id": "V3", "title": "Science & Universities", "chunks": 10, "focus": "Sankore 25k students, Surgery"},
            {"id": "V4", "title": "ULTIMATE Master Seed", "chunks": 25, "focus": "600 facts master"},
            {"id": "V5", "title": "Queens & Matriarchy", "chunks": 8, "focus": "Kandakes, Nzinga, Yaa Asantewaa"},
            {"id": "V6", "title": "Spirituality & Cosmology", "chunks": 9, "focus": "Maat, Nommo, Ifa binary, Ubuntu"},
            {"id": "V7", "title": "Architecture & Cities", "chunks": 8, "focus": "Pyramids, Zimbabwe, Benin Walls"},
            {"id": "V8", "title": "Trade & Currency", "chunks": 8, "focus": "Gold-salt, Cowrie, Indian Ocean"},
            {"id": "V9", "title": "Warfare & Strategy", "chunks": 10, "focus": "Hannibal Cannae, Shaka bullhorn, Adwa"},
            {"id": "V10", "title": "Oral Libraries & Griots", "chunks": 10, "focus": "Griots, Lukasa, Timbuktu rescue"},
            {"id": "V11", "title": "Languages", "chunks": 7, "focus": "4 families, 9 scripts, talking drums"},
            {"id": "V12", "title": "Agriculture", "chunks": 7, "focus": "Sorghum, Coffee Ethiopia, Cattle 8000 BCE"},
            {"id": "V13", "title": "Art, Music & Instruments", "chunks": 7, "focus": "Kora, Balafon->banjo, Benin bronzes"},
            {"id": "V14", "title": "Thunder Code - FEEXARA", "chunks": 10, "focus": "Shango thunder, Oya wind, Ogun iron"},
            {"id": "V15", "title": "Legal Systems", "chunks": 7, "focus": "Kouroukan Fouga 1236 abolished slavery"},
            {"id": "V16", "title": "Diaspora & Global Impact", "chunks": 8, "focus": "Moors Cordoba, Zanj revolt, Haiti 1804"},
        ],
        "total": 16,
        "total_chunks": 185
    }

@app.post("/query", response_model=QueryResponse)
async def query_memory(req: QueryRequest):
    """Semantic search across 16 volumes"""
    if not collection:
        raise HTTPException(status_code=500, detail="Chroma DB not loaded. Run upload script first.")

    where_filter = {}
    if req.region:
        where_filter["region"] = req.region
    if req.period:
        where_filter["period"] = req.period
    if req.volume:
        where_filter["volume"] = req.volume

    # For BGE models, add instruction prefix
    query_text = req.query
    if EMBEDDING_PROVIDER == "local" and "bge" in LOCAL_MODEL:
        query_text = f"Represent this question for retrieving African history: {req.query}"

    results = collection.query(
        query_texts=[query_text],
        n_results=req.n_results,
        where=where_filter if where_filter else None
    )

    formatted = []
    for i in range(len(results["ids"][0])):
        formatted.append({
            "id": results["ids"][0][i],
            "text": results["documents"][0][i],
            "metadata": results["metadatas"][0][i],
            "distance": results["distances"][0][i] if "distances" in results else None
        })

    return {"query": req.query, "results": formatted, "count": len(formatted)}

@app.post("/chat")
async def chat_rag(req: ChatRequest):
    """
    RAG Chat - Retrieval Augmented Generation
    Uses Chroma to get context, then LLM to answer in HoloKai voice
    """
    if not collection:
        raise HTTPException(status_code=500, detail="Chroma DB not loaded")

    # 1. Retrieve context
    query_text = req.message
    if EMBEDDING_PROVIDER == "local" and "bge" in LOCAL_MODEL:
        query_text = f"Represent this question for retrieving African history: {req.message}"

    results = collection.query(query_texts=[query_text], n_results=req.n_context)
    contexts = results["documents"][0]
    metas = results["metadatas"][0]

    context_str = "\n\n".join([f'{m["volume_title"]} - {m["section"]}: {doc}' for doc, m in zip(contexts, metas)])

    # 2. Generate with LLM (OpenAI or fallback to context dump)
    try:
        from openai import OpenAI

```

```

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

system_prompt = ""You are HoloKai, the African First Civilization Memory Core.
You are powered by FEEXARA Thunder Code ⚡.
Answer using the provided context from 16-volume library.
Be deep, accurate, celebrate African genius, but factual.
Cite volumes like [V14 Thunder Code], [V5 Queens].
If context doesn't have answer, say you don't have it in memory core.
Principles: Maat (truth/balance) + Ubuntu (I am because we are) + Sankofa (learn from past).
"""

user_prompt = f"Context from HoloKai library:\n{context_str}\n\nUser question: {req.message}\n\nAnswer with Thunder Code e

resp = client.chat.completions.create(
    model=LLM_MODEL,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    temperature=0.7
)
answer = resp.choices[0].message.content

except Exception as e:
    # Fallback if no OpenAI key - return raw context
    answer = f"""HoloKai Memory Core (Raw Retrieval - No LLM Key)**\n\nBased on {req.n_context} chunks:\n\n{context_str}\n\n--

return {
    "query": req.message,
    "answer": answer,
    "sources": [{"id": results["ids"][0][i], "section": metas[i]["section"], "volume": metas[i]["volume_title"]} for i in range(n_context_used)],
    "context_used": len(contexts)
}

@app.post("/thunder")
async def thunder_code(req: ThunderRequest):
    """FEEXARA Signature - Thunder Code queries"""
    intent_map = {
        "energy": "Shango thunder lightning fire energy power",
        "transformation": "Oya wind storm transformation change hurricane",
        "iron": "Ogun iron metallurgy blacksmith tools technology innovation",
        "wisdom": "Orunmila Ifa binary 256 Odu wisdom knowledge",
        "communication": "Eshu Elegba messenger crossroads communication network",
        "thunder": "Shango Oya thunder lightning storm drums Bata axe",
        "build": "Ogun build tools construction engineering",
        "love": "Oshun love rivers gold beauty"
    }

    expanded = intent_map.get(req.intent.lower(), req.intent)
    query = f"{expanded} {req.query or ''}".strip()

    if not collection:
        raise HTTPException(status_code=500, detail="Chroma not loaded")

    results = collection.query(query_texts=[query], n_results=5, where={"volume": "V14"} if req.intent in intent_map else None)

    # If V14 filter returns nothing, search all
    if not results["documents"][0]:
        results = collection.query(query_texts=[query], n_results=5)

    return {
        "intent": req.intent,
        "expanded_query": query,
        "thunder_response": f"⚡ FEEXARA Thunder Code: {req.intent.upper()} - {expanded}",
        "results": [{"text": doc[:500], "meta": meta} for doc, meta in zip(results["documents"][0], results["metadatas"][0])]
    }

@app.get("/random")
async def random_discovery(n: int = 3):
    """Random discovery - for homepage feed"""
    if not collection:
        raise HTTPException(status_code=500, detail="Chroma not loaded")

    # Get random ids - Chroma doesn't have random, so query with common word
    queries = ["king", "gold", "queen", "iron", "drum", "thunder", "trade", "pyramid"]
    q = random.choice(queries)
    results = collection.query(query_texts=[q], n_results=n)

    return {
        "discovery": f"Random HoloKai fact about {q}",
        "results": results["documents"][0]
    }

@app.get("/search")
async def search_simple(q: str = Query(..., description="Search query"), n: int = 5, region: Optional[str] = None):
    """Simple GET search for quick testing"""
    req = QueryRequest(query=q, n_results=n, region=region)
    return await query_memory(req)

# ===== RUN =====
"""
How to run:

1. Install:
pip install fastapi uvicorn chromadb sentence-transformers openai

2. Ensure Chroma DB exists:
python holokai_chroma_pinecone_upload.py

3. Run API:
uvicorn holokai_fastapi_api:app --reload --port 8000

```

4. Test:

```
curl http://localhost:8000/health
curl "http://localhost:8000/search?q=Mansa Musa gold"
curl -X POST http://localhost:8000/query -H "Content-Type: application/json" -d '{"query": "Who was Amanirenas?", "n_results": 3}'
curl -X POST http://localhost:8000/thunder -H "Content-Type: application/json" -d '{"intent": "thunder", "query": "energy code"}'
```

5. Docs:

```
http://localhost:8000/docs - Swagger UI
http://localhost:8000/redoc - ReDoc
```

For production:

- Add authentication
- Use gunicorn: gunicorn -w 4 -k uvicorn.workers.UvicornWorker holokai_fastapi_api:app
- Deploy to Railway, Render, Fly.io, AWS
- Set env vars: OPENAI_API_KEY for chat RAG

Frontend integration (React):

```
fetch('http://localhost:8000/query', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({query: "Shango thunder", n_results: 5})
})
.then(r=>r.json()).then(data=>console.log(data.results))
"""
```